

Московский Физико-Технический Институт
Физтех-школа Аэрокосмических технологий
Институт Аэромеханики и Летательной Техники



Архитектура Компьютера и Операционные Системы.

Часть 2. Основы операционных систем

Лекция 4: Потоки ввода-вывода, pipe

Новиков Андрей Валерьевич
д.ф.-м.н.

Жуковский

Потоки ввода-вывода

❑ Передача данных по каналам связи

- потокковая модель
- модель сообщений

❑ Потокковая модель передачи данных

- операции приёма/передачи (ввода/вывода) не интересуются содержимым
- данные считаются непрерывным потоком байт без внутренней структуры

❑ Потокковая передача

- между процессами
- между процессом и диском (файлом)

С-функции для работы с файлами

```
#include <stdio.h>
```

```
FILE* fopen(const char* filename, const char* mode);
```

```
int fclose(FILE* stream);
```

```
int fscanf(FILE* stream, const char* format,...);
```

```
int fprintf(FILE* stream, const char* format, ...);
```

```
char* fgets(char* str, int num, FILE* stream);
```

```
size_t fread(void* data, size_t item_size, size_t count, FILE* stream);
```

```
size_t fwrite(void* data, size_t item_size, size_t count, FILE* stream);
```

С-функции для работы с файлами

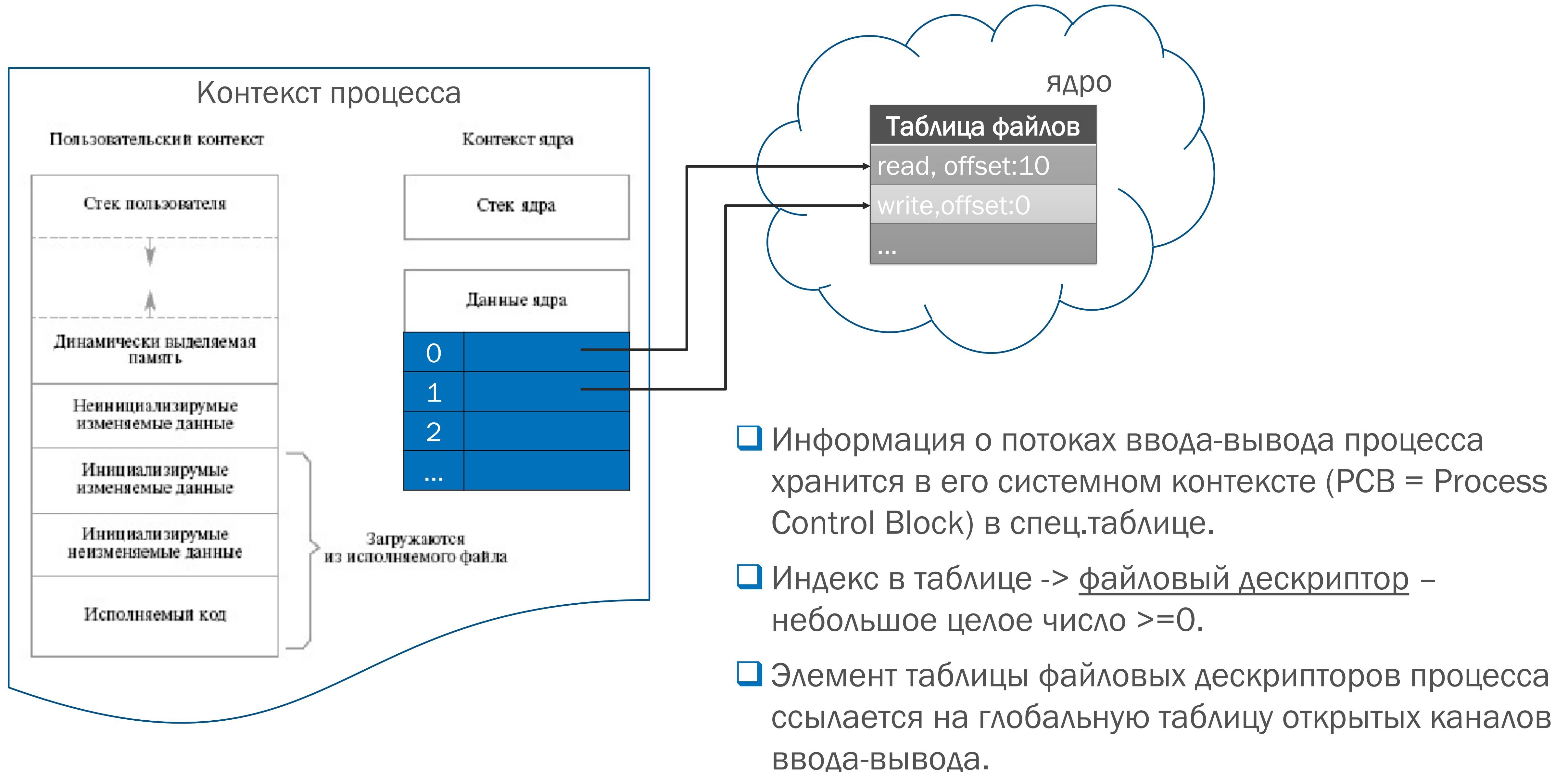
```
#include <stdio.h>
FILE* stream;
fopen(...), fclose(...);
fgets(...), fscanf(...), fprintf(...);
fread(...), fwrite(...);
```



```
#include <fcntl.h>
int file_descriptor;
open(...), close(...);
read(...), write(...);
```

- ❑ Надстройка (сервисный интерфейс) над системными вызовами
- ❑ С-функции оперируют объектом FILE (структура)
- ❑ Системные вызовы оперируют с файловыми дескрипторами (целое число, идентификатор)

Файловый дескриптор



Файловый дескриптор

- ❑ Файловый дескриптор — небольшое целое неотрицательное число, которое для текущего процесса, в данный момент времени однозначно определяет действующий канал ввода-вывода.

- ❑ Файловые дескрипторы, создаваемые по умолчанию
 - 0 (STDIN_FILENO) – стандартный поток ввода (*по умолчанию клавиатура*)
 - 1 (STDOUT_FILENO) – стандартный поток вывода (экран)
 - 2 (STDERR_FILENO) – стандартный поток ошибок (экран)

Открытие потока (файла)

```
#include <fcntl.h>

int open(const char* path,
         int flags, int mode);
```

- ❑ Возвращаемое значение
 - >=0 – файловый дескриптор
 - 1 – ошибка + код ошибки в переменной **errno**
- ❑ Параметр **path** – путь (полное имя) файла
- ❑ Параметр **flags** – комбинация с помощью `|` (битовое или) битовых констант:
 - Обязательный флаг режима доступа к файлу
 - O_RDONLY (00000) – только чтение
 - O_WRONLY (00001) – только запись
 - O_RDWR (00002) – чтение/запись

Открытие потока (файла)

```
#include <fcntl.h>

int open(const char* path,
         int flags, int mode);
```

❑ ... flags

- **Необязательные флаги создания**
 - O_CREAT (00100) – создать если не существует
 - O_EXCL (00200) – вместе с предыдущим, создавать эксклюзивно, ошибка если существует
 - O_TRUNC (01000) – при открытии удалить содержимое
 - ...
- **Необязательные флаги статуса файла**
 - O_APPEND (02000) – писать в конец
 - O_NDELAY или O_NONBLOCK – для специальным каналам связи, не ожидать открытия и завершения последующих операций
 - O_SYNC – все последующие операции записи будут ожидать физической записи
 - ...

Открытие потока (файла)

```
#include <sys/stat.h>
#include <fcntl.h>

int open(const char* path,
         int flags, int mode);
```

□ **mode** – права доступа к создаваемому файлу, игнорируются если файл существует, комбинация “битовым или” констант

- S_IRUSR (0400) – чтение для владельца
- S_IWUSR (0200) – запись для владельца
- S_IXUSR (0100) – исполнение для владельца
- S_IRGRP (0040) – чтение для группы
- S_IWGRP (0020) – запись для группы
- ...
- S_IROTH (0004) – чтение для остальных
- ...

Открытие файла. Пример

```
#include <fcntl.h>

int main(int argc, char* argv[], char* envp[])
{
    int fd = open("test.txt", O_WRONLY|O_CREAT|O_EXCL, 0664);
    if( fd == -1 ){ perror("open"); return -1; }

    // write(fd,...);
    return 0;
}
```


Запись/чтение из потока

```
#include <sys/types.h>
#include <unistd.h>

size_t read(int fd, void* ptr,
            size_t nbytes);

size_t write(int fd, void* ptr,
             size_t nbytes);
```

- ❑ Возвращаемое значение
 - ≥ 0 – количество считанных/записанных байт (может быть меньше ожидаемого)
 - 1 – ошибка
- ❑ **fd** – файловый дескриптор
- ❑ **ptr** – указатель в памяти, начиная с которого размещается информация при чтении или берётся информация при записи
- ❑ **nbytes** – количество байт, которые нужно считать/записать

Чтение из потока. Пример

```
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char* argv[], char* envp[])
{
    int fd = open("test.txt", O_RDONLY);
    if( fd == -1 ){ perror("open"); return -1; }

    const int len = 128;
    char data[len];
    if( read(fd, data, len) < len )
        ; //
    return 0;
}
```


Заккрытие потока

```
#include <unistd.h>
```

```
int close(int fd);
```

- ❑ Возвращаемое значение
 - 0 – успешное завершение
 - 1 – ошибка
- ❑ Параметр **fd** – файловый дескриптор

Потоковая передача через «безымянный канал» `pipe`

- ❑ `pipe` — «труба», «канал», «конвейер» -- потоковая передача между родственными процессами.
- ❑ При чтении информация из «трубы» удаляется (!).
- ❑ `pipe` – некая область памяти в ядре в виде кольцевого буфера, куда можно временно поместить данные и затем считать в порядке FIFO (first-in-first-out).



Создание «безымянного канала»

```
#include <unistd.h>

int pipe(int fd[2]);
```



- ❑ Создаёт «pipe» внутри ядра. Размер предопределён и не регулируется пользователем.
- ❑ Параметр **fd[2]** – массив из 2х значений, заполняемый при успешном вызове
 - **fd[0]** – файловый дескриптор для **чтения** (**ВЫХОД** трубы)
 - **fd[1]** – файловый дескриптор для **записи** (**ВХОД** трубы)
- ❑ Возвращает
 - 0 – успешное создание
 - -1 – ошибка

pipe. Пример

```
#include <unistd.h>

int main(int argc, char* argv[], char* envp[])
{
    int fd[2];
    if( pipe(fd) == -1 ){ perror("pipe"); return -1; }

    char out[] = "Hello world";
    write(fd[1], out, sizeof(out));

    char in[128];
    read(fd[0], in, sizeof(in));
    //
    return 0;
}
```

Особенности read() из `pipe`

Ситуация	Поведение
Попытка прочитывать меньше байт, чем есть в наличии в канале связи.	Читает требуемое количество байт и возвращает значение, соответствующее прочитанному количеству. Прочитанная информация удаляется из канала связи.
В канале связи находится меньше байт, чем затребовано, но не нулевое количество.	Читает все, что есть в канале связи, и возвращает значение, соответствующее прочитанному количеству. Прочитанная информация удаляется из канала связи.
Попытка читать из канала связи, в котором нет информации. Блокировка вызова разрешена.	Вызов блокируется до тех пор, пока не появится информация в канале связи и пока существует процесс, который может передать в него информацию. Если информация появилась, то процесс разблокируется, и поведение вызова определяется двумя предыдущими строками таблицы. Если в канал некому передать данные (нет ни одного процесса, у которого этот канал связи открыт для записи), то вызов возвращает значение 0. Если канал связи полностью закрывается для записи во время блокировки читающего процесса, то процесс разблокируется, и системный вызов возвращает значение 0.
Попытка читать из канала связи, в котором нет информации. Блокировка вызова не разрешена.	Если есть процессы, у которых канал связи открыт для записи, системный вызов возвращает значение -1 и устанавливает переменную errno в значение EAGAIN. Если таких процессов нет, системный вызов возвращает значение 0.

Особенности write() в `pipe`

Ситуация	Поведение
Попытка записать в канал связи меньше байт, чем осталось до его заполнения.	Требуемое количество байт помещается в канал связи, возвращается записанное количество байт.
Попытка записать в канал связи больше байт, чем осталось до его заполнения. Блокировка вызова разрешена.	Вызов блокируется до тех пор, пока все данные не будут помещены в канал связи. Если размер буфера канала связи меньше, чем передаваемое количество информации, то вызов тем самым будет ждать, пока часть информации не будет считана из канала связи. Возвращается записанное количество байт.
Попытка записать в канал связи больше байт, чем осталось до его заполнения, но меньше, чем размер буфера канала связи. Блокировка вызова запрещена.	Системный вызов возвращает значение -1 и устанавливает переменную errno в значение EAGAIN.
В канале связи есть место. Попытка записать в канал связи больше байт, чем осталось до его заполнения, и больше, чем размер буфера канала связи. Блокировка вызова запрещена.	Записывается столько байт, сколько осталось до заполнения канала. Системный вызов возвращает количество записанных байт.
Попытка записи в канал связи, в котором нет места. Блокировка вызова не разрешена.	Системный вызов возвращает значение -1 и устанавливает переменную errno в значение EAGAIN.
Попытка записи в канал связи, из которого некому больше читать, или полное закрытие канала на чтение во время блокировки системного вызова.	Если вызов был заблокирован, то он разблокируется. Процесс получает сигнал SIGPIPE. Если этот сигнал обрабатывается пользователем, то системный вызов вернет значение -1 и установит переменную errno в значение EPIPE.

Потоковая передача через «именованный канал» FIFO

- ❑ **FIFO** или **named pipe** — именованная «труба», «канал», «конвейер» -- потоковая передача между любыми процессами.
- ❑ При чтении информации из «трубы» она удаляется(!).
- ❑ **FIFO** – некая область памяти в ядре в виде кольцевого буфера, куда можно временно поместить данные и затем считать в порядке FIFO (first-in-first-out).
- ❑ FIFO даётся имя путём создания «метки» в файловой системе.



Создание именованного канала

```
$ mkfifo mylabel.fifo
```

```
#include <fcntl.h>
#include <stdio.h>

int main(int argc, char* argv[], char* envp[])
{
    int fd = open("mylabel.fifo", O_RDONLY);
    if( fd < 0 ){ perror("open"); return 1; }

    const int len = 128;
    char data[len];
    if( read(fd, data, len) < len )
        do_something_with(data);
    return 0;
}
```